

IMAGE PROCESSING AND ITS APPLICATION

MODULE - 3

Prof. Nishchitha MH

Assistant Professor

Dept. of Robotics and Artificial Intelligence

Fundamentals of Deep Learning

Deep Learning is a subset of **Machine Learning** that uses **Artificial Neural Networks (ANNs)** with multiple hidden layers to automatically learn patterns from data.

Types of Deep Learning Networks

1. Artificial Neural Networks (ANN)

Basic fully connected networks.

2. Convolutional Neural Networks (CNN)

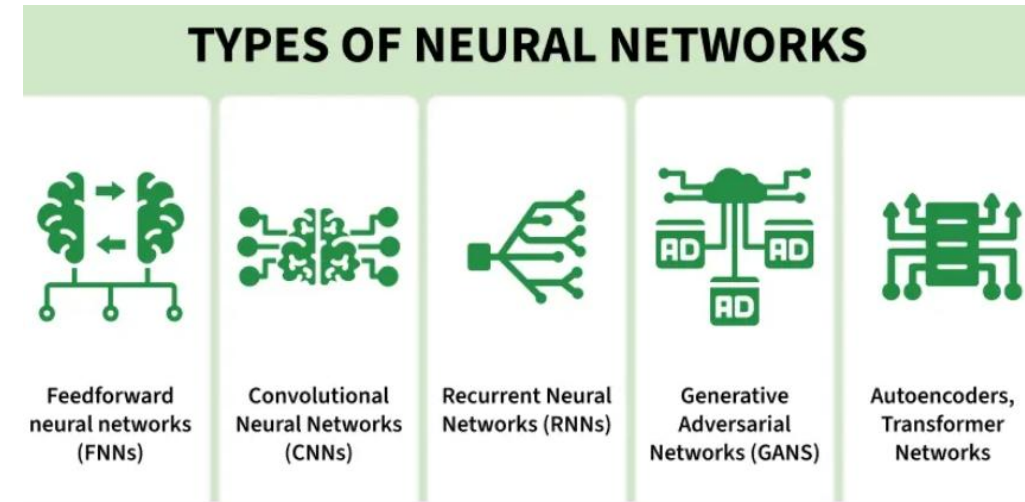
Used for image processing and computer vision.

3. Recurrent Neural Networks (RNN)

Used for sequential data like text and speech.

4. Transformers

Used in NLP and large language models.



Neural Network

A **Neural Network** consists of interconnected nodes called **neurons** arranged in layers:

Input Layer

Hidden Layer(s)

Output Layer

$$y = f \left(\sum w_i x_i + b \right)$$

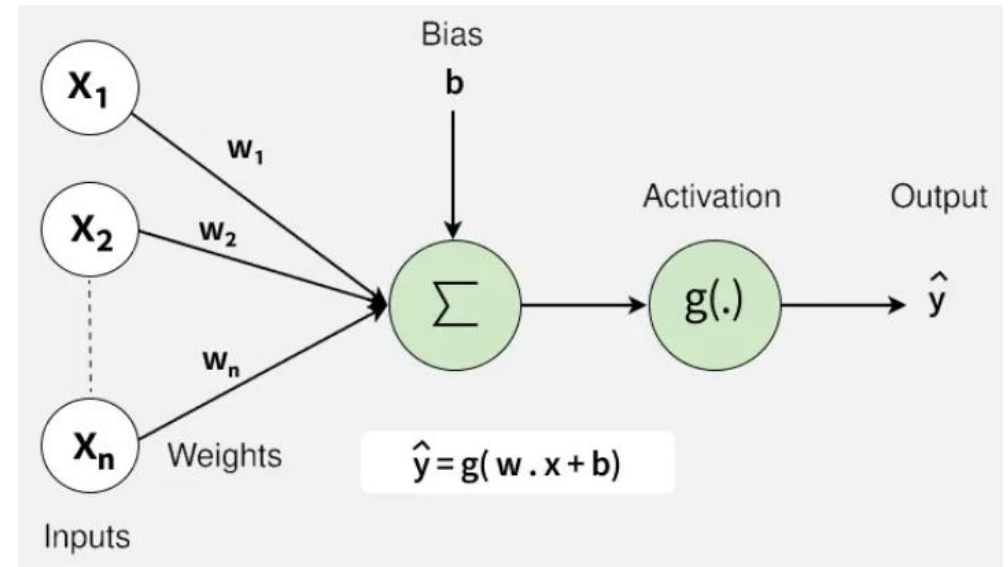
Where:

$x_i \rightarrow$ inputs

$w_i \rightarrow$ weights

$b \rightarrow$ bias

$f \rightarrow$ activation function



Key Components of Deep Learning

- **Neurons:** The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections:** Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases:** These parameters determine the strength and influence of connections.
- **Propagation Functions:** Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule:** The method that adjusts weights and biases over time to improve accuracy.

Key Components of Deep Learning

1. Weights

Control importance of input features
Updated during training

2. Bias

Shifts activation function

3. Activation Functions

Introduce non-linearity.

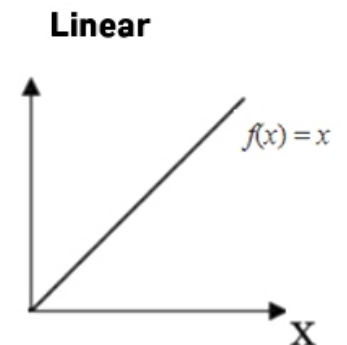
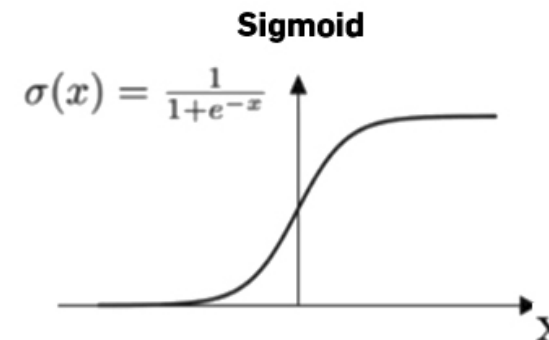
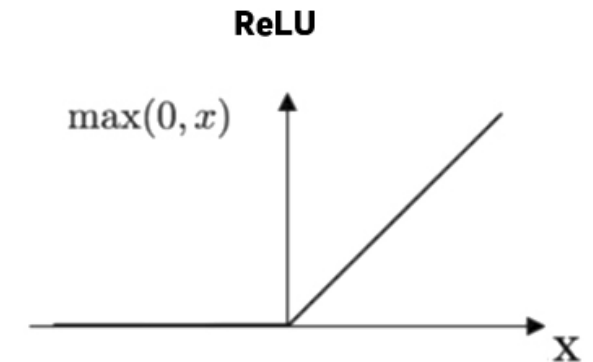
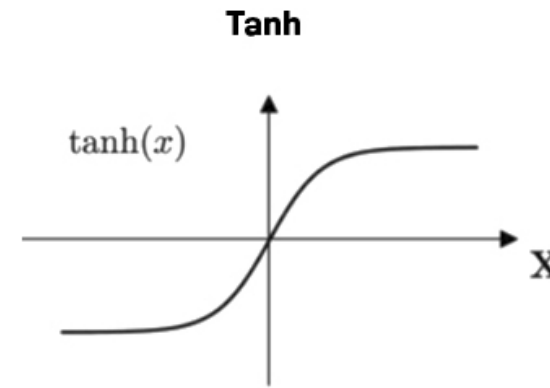
Common activation functions:

Sigmoid

ReLU (Rectified Linear Unit)

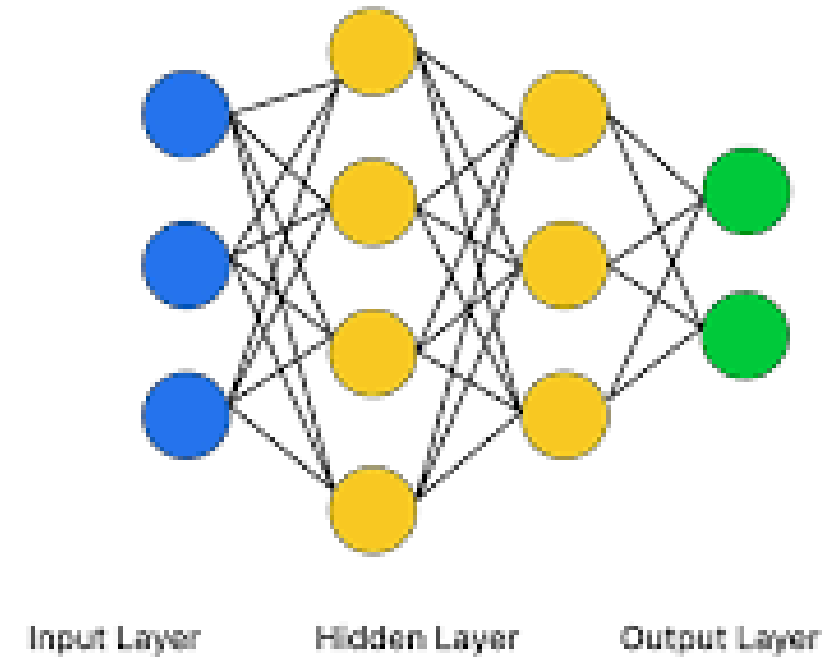
Tanh

Softmax



Layers in Neural Network Architecture

- **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
- **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.
- **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.



Training Process

Deep learning models learn using:

1. Forward Propagation

Input → Hidden layers → Output

2. Loss Function

Measures error between predicted and actual output.

Examples:

Mean Squared Error (MSE)

Cross-Entropy Loss

3. Backpropagation

Computes gradients

Updates weights using **Gradient Descent**

Forward Propagation

When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation. Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

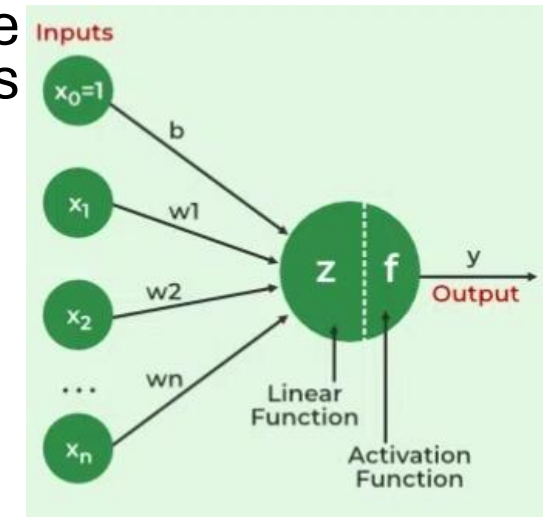
where

w represents the weights

x represents the inputs

b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the system, **enabling the network to learn more complex patterns**. Popular activation functions include ReLU, sigmoid and tanh.



Backpropagation

After forward propagation, the network evaluates its performance using a loss function which measures the difference between the actual output and the predicted output. The goal of training is to minimize this loss. This is where backpropagation comes into play:

Loss Calculation:

The network calculates the loss which provides a **measure of error in the predictions**. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.

Gradient Calculation:

The network computes the gradients of the **loss function with respect to each weight and bias in the network**. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.

Weight Update:

Once the gradients are calculated, the weights and biases are updated using an **optimization algorithm**. The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

Iteration

This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.

Applications of Deep Learning

- Image classification
- Object detection
- Speech recognition
- Natural language processing
- Medical diagnosis
- Autonomous vehicles

Deep Learning

Advantages

- High accuracy
- Learns complex patterns
- Automatic feature extraction

Limitations

- Requires large dataset
- High computational cost
- Needs GPUs
- Less interpretable

Basic Workflow of Deep Learning

- Data Collection
- Data Preprocessing
- Model Design
- Training
- Evaluation
- Deployment

Hyperparameters

- **Hyperparameters** are configuration values that are set **before training** a deep learning model.
- They control the learning process and model structure.

Parameters:

- Learned during training
- Weights and biases
- Updated by backpropagation

Hyperparameters:

- Set before training
- Learning rate, batch size, etc.
- Not directly updated by model

Types of Hyperparameters

Model Hyperparameters

These define the **structure of the neural network**.

1. Number of Hidden Layers
2. Number of Neurons per Layer
3. Activation Function
4. Dropout Rate

Training Hyperparameters

These control the **learning process**.

1. Learning Rate (α)
2. Batch Size
3. Number of Epochs
4. Optimizer
5. Momentum
6. Weight Initialization
7. Regularization Parameter (λ)

Model Hyperparameters

1. Number of Hidden Layers

More layers → deeper network

Too many → overfitting

2. Number of Neurons per Layer

Controls model capacity

More neurons → more complexity

3. Activation Function

ReLU, Sigmoid, Tanh, Softmax

4. Dropout Rate

Fraction of neurons randomly disabled

Prevents overfitting

Training Hyperparameters

Learning Rate (α)

$$w = w - \alpha \frac{\partial L}{\partial w}$$

$w \rightarrow$ weight

$\alpha \rightarrow$ learning rate

$L \rightarrow$ loss function

$\frac{\partial L}{\partial w} \rightarrow$ gradient of loss with respect to weight

- Small learning rate $\rightarrow$ slow training
- Large learning rate $\rightarrow$ unstable training

Training Hyperparameters

Batch Size

The number of training samples processed in one forward/backward pass before the model's internal parameters (weights) are updated

- Types:
 - Batch Gradient Descent
 - Mini-batch Gradient Descent
 - Stochastic Gradient Descent
- Small batch → noisy but better generalization
- Large batch → stable but needs more memory

Training Hyperparameters

Number of Epochs

Number of complete passes through the dataset.

- Too few → underfitting
- Too many → overfitting

Optimizer

Algorithm used to update weights.

- SGD - Stochastic Gradient Descent
- Adam
- RMSprop

Training Hyperparameters

Momentum

Helps accelerate gradient descent.

Weight Initialization

Initial values of weights.

Poor initialization → slow convergence.

Regularization Parameter (λ)

Controls overfitting

Hyperparameter Tuning Methods

Selecting the best hyperparameters is called **hyperparameter tuning**.

1. Manual Search

Trial and error.

2. Grid Search

Try all combinations.

3. Random Search

Random combinations.

4. Bayesian Optimization

Smarter probabilistic search.

Importance of Hyperparameters

- Affect accuracy
- Affect training speed
- Control overfitting
- Improve model performance

Problem	Cause
Slow training	Low learning rate
Divergence	High learning rate
Overfitting	Too many epochs
Underfitting	Small model

Batch Size in Deep Learning

- **Batch size** is the number of training samples processed by the neural network **before updating the model weights**.
- During training, the dataset is divided into smaller groups called **batches**, and the model processes one batch at a time.
- Example:
If a dataset has **1000 images** and the **batch size = 100**, then the model will process **100 images at a time**.

Relationship Between Batch Size, Epochs, and Iterations

Epoch → One complete pass through the entire dataset

Batch Size → Number of samples processed before weight update

Iterations → Number of batches required to complete one epoch

Formula:

$$\text{Iterations} = \frac{\text{Total number of samples}}{\text{Batch size}}$$

Example:

Dataset = 1000 samples

Batch size = 100

$$\text{Iterations} = \frac{1000}{100} = 10$$

So 10 iterations = 1 epoch.

Types of Gradient Descent Based on Batch Size

- **Batch Gradient Descent:** Uses **entire dataset** for weight update.
- Example: Batch size = total dataset
- Advantages:
 - Stable gradient
- Disadvantages:
 - Very slow for large datasets
 - High memory requirement

Types of Gradient Descent Based on Batch Size

- **Stochastic Gradient Descent (SGD):** Updates weights **after every single sample.**
 - Batch size = 1
- **Advantages:**
 - Faster updates
 - Escapes local minima
- **Disadvantages:**
 - Very noisy updates

Types of Gradient Descent Based on Batch Size

- **Mini-Batch Gradient Descent:** Most commonly used method.
 - Batch size = small group (e.g., 32, 64, 128)
- Advantages:
 - Faster training
 - Efficient GPU usage
 - Balanced stability and speed

Effect of Batch Size on Training

Small Batch Size

- Example: 8, 16, 32
- Advantages:
 - Better generalization
 - Less memory usage
 - More frequent updates
- Disadvantages:
 - Noisy gradient
 - Slower convergence

Large Batch Size

- Example: 256, 512, 1024
- Advantages:
 - Stable gradient updates
 - Faster computation on GPUs
- Disadvantages:
 - Requires more memory
 - Risk of poor generalization

Choosing the Right Batch Size

- Common values used in practice: **16, 32, 64, 128**
- Most deep learning models use **32 or 64** as default batch size.
- Choice depends on:
 - GPU memory
 - Dataset size
 - Model complexity

Example Training Process

Dataset = 1000 images

Batch size = 50

Steps:

1. Model processes first 50 images
2. Compute loss
3. Backpropagation updates weights
4. Move to next 50 images
5. Repeat until all 1000 images are processed

This completes **one epoch**.

Batch Size in Deep Learning

Advantages

- Reduces memory usage
- Speeds up training
- Efficient GPU utilization
- Stable learning

Disadvantages

- Requires tuning
- Large batch sizes may reduce model generalization

Learning Rate in Deep Learning

- The **learning rate** is a hyperparameter that determines **how much the model's weights are updated during training**. It controls the **step size** taken while minimizing the loss function using optimization algorithms such as gradient descent.
- The learning rate decides **how fast or slow a neural network learns**.
- The learning rate controls:
 - Speed of training
 - Stability of the model
 - Convergence to optimal solution

$$w = w - \alpha \frac{\partial L}{\partial w}$$

$w \rightarrow$ weight

$\alpha \rightarrow$ learning rate

$L \rightarrow$ loss function

$\frac{\partial L}{\partial w} \rightarrow$ gradient of loss with respect to weight

Effect of Learning Rate

Small Learning Rate

Example: 0.0001

Advantages:

- Stable learning

- Precise convergence

Disadvantages:

- Very slow training

Large Learning Rate

Example: 0.1

Advantages:

- Faster learning

Disadvantages:

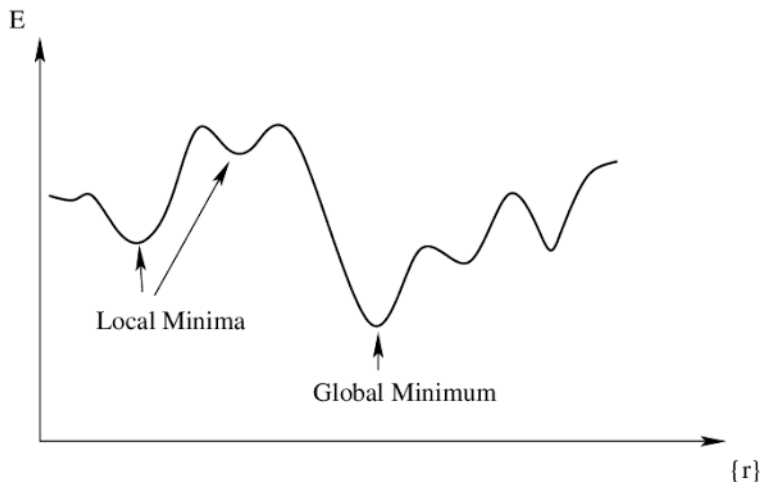
- Overshooting the optimal solution

- Training becomes unstable

- Loss may diverge

Ideal Learning Rate

- A **moderate learning rate** helps the model converge efficiently.
- Common values used in practice: 0.1, 0.01, 0.001, 0.0001
- Most deep learning models use **0.001** as a starting point.
- If learning rate is **too small**, training takes longer.
- If learning rate is **too large**, the algorithm may never converge.
- The goal is to **reach the global minimum of the loss function efficiently**, minimum possible error across the entire dataset



Learning Rate Schedules

Instead of keeping the learning rate constant, it can be **adjusted during training**.

1. **Step Decay:** Learning rate decreases after fixed number of epochs.
2. **Exponential Decay:** Learning rate gradually decreases exponentially.
3. **Time-Based Decay:** Learning rate decreases as training progresses.
4. **Adaptive Learning Rates:** Optimizers automatically adjust learning rate.

Importance of Learning Rate

- Learning rate affects:
 - Training speed
 - Model accuracy
 - Stability of training
 - Convergence of optimization
- A poorly chosen learning rate can cause **training failure**.

Advantages of Proper Learning Rate

- Faster convergence
- Stable training
- Better model performance

Disadvantages of Poor Learning Rate

- Slow training
- Oscillation around minimum
- Diverging loss

Activation Function

- An **activation function** is a mathematical function applied to the output of a neuron in a neural network. It determines **whether a neuron should be activated or not** by introducing **non-linearity** into the model.
- Without activation functions, a neural network would behave like a **simple linear model** and would not be able to learn complex patterns.
- The activation function f transforms the weighted sum into a useful output.

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

Types of Activation Functions

- Linear Activation Function

The output is directly proportional to the input.

$$f(x) = x$$

Characteristics

Simple

No non-linearity

Rarely used in deep learning

Limitation

Stacking multiple linear layers still results in a **linear function**.

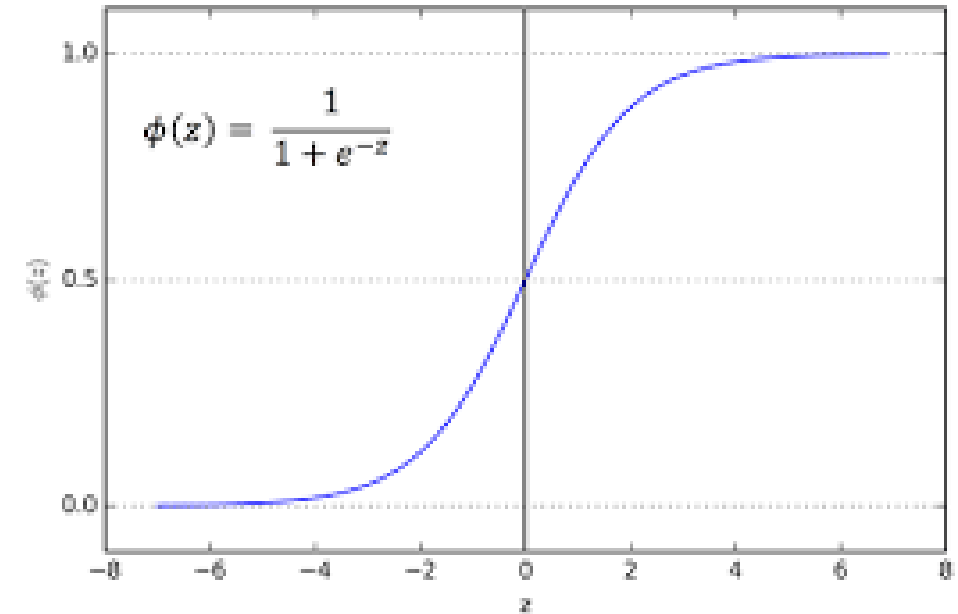
- Non-Linear Activation Functions

- Sigmoid Function
- Tanh Function
- ReLU (Rectified Linear Unit)
- Softmax Function

Sigmoid Function

- The **Sigmoid function** is a type of **activation function** used in neural networks. It transforms the input value into a **value between 0 and 1**, making it useful for models that need to output probabilities.
- The output of the sigmoid function lies between:
$$0 < f(x) < 1$$
- This makes it suitable for **binary classification problems**.
 - Large negative input → output close to **0**
 - Input around **0** → output close to **0.5**
 - Large positive input → output close to **1**

Input	Output
$-\infty$	0
0	0.5
$+\infty$	1



$x \rightarrow$ input value

$e \rightarrow$ Euler's number (≈ 2.718)

$f(x) \rightarrow$ output of the sigmoid function

$$x = 0$$
$$f(0) = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

Sigmoid Function

- Advantages

- Smooth and continuous function
- Output values between **0 and 1**
- Useful for **probability interpretation**
- Easy derivative calculation

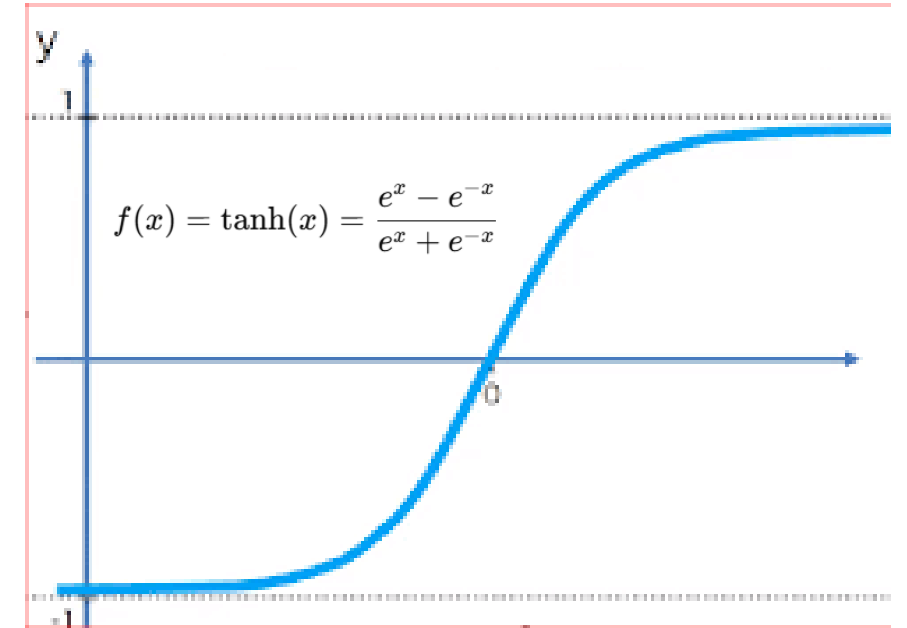
- Disadvantages

- **Vanishing Gradient Problem**
Gradients become very small for large inputs, slowing training.
- **Not Zero-Centered**
Output values are always positive.
- **Slow Convergence**
Training becomes slower compared to modern activation functions like ReLU.

Tanh Function (Hyperbolic Tangent)

- The **Tanh (Hyperbolic Tangent) function** is a non-linear **activation function** used in neural networks. It transforms input values into outputs ranging from **-1 to +1**.
- Tanh is similar to the sigmoid function but produces **zero-centered outputs**, which often improves learning in neural networks.
- The output values of tanh lie between:
$$-1 < f(x) < 1$$

Input	Output
$-\infty$	-1
0	0
$+\infty$	1



$x \rightarrow$ input value

$e \rightarrow$ Euler's number (≈ 2.718)

$f(x) \rightarrow$ output of tanh function

$$x = 0$$

$$\tanh(0) = 0$$

Tanh Function (Hyperbolic Tangent)

- **Advantages of Tanh Function**

- Output is **zero-centered**
- Stronger gradients than sigmoid
- Faster convergence during training
- Works well for hidden layers

- **Disadvantages**

- **Vanishing Gradient Problem**
For large input values, gradients become very small.
- **Computationally Expensive**
Uses exponential functions.
- **Less popular than ReLU** in modern deep learning.

ReLU (Rectified Linear Unit)

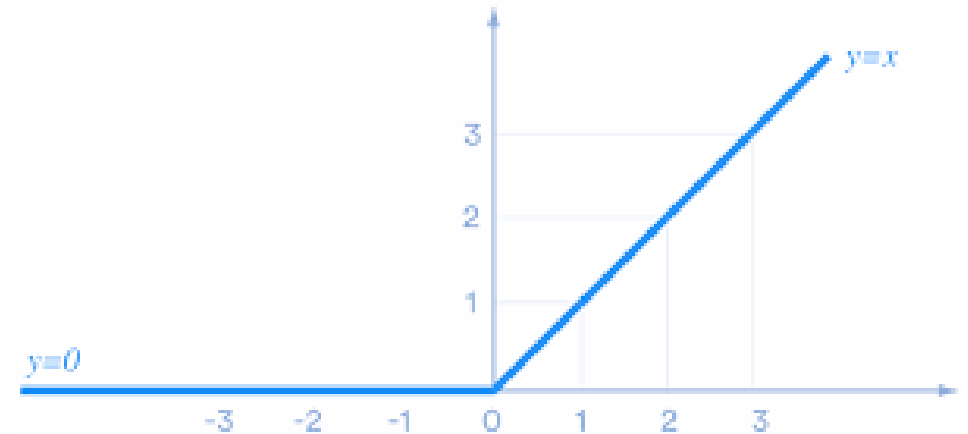
- The **ReLU (Rectified Linear Unit)** is one of the most widely used **activation functions** in deep learning. It introduces **non-linearity** into neural networks and helps models learn complex patterns efficiently.
- ReLU outputs the input directly if it is positive; otherwise, it outputs **zero**.

$$0 \leq f(x) < \infty$$

Negative inputs $\rightarrow 0$

Positive inputs $\rightarrow$ **same value**

$$f(x) = \max(0, x)$$



$$f(x) = \begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$$

ReLU (Rectified Linear Unit)

Advantages of ReLU

- **Computationally Efficient**
Requires only a simple comparison.
- **Reduces Vanishing Gradient Problem**
Gradients remain strong for positive values.
- **Faster Training**
Networks converge faster.
- **Sparse Activation**
Many neurons output zero, improving efficiency.

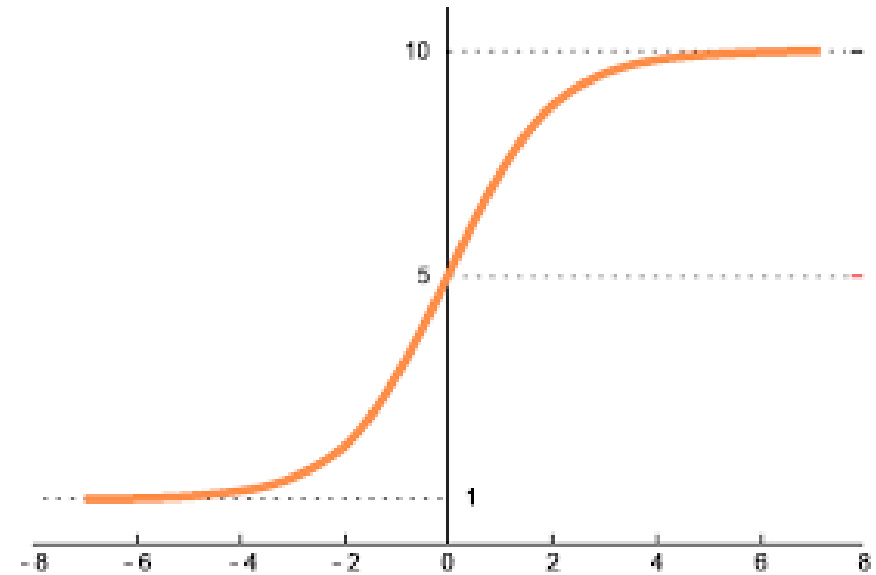
Disadvantages

- **Dead Neuron Problem**
If inputs become negative, neurons may always output zero.
- **Unbounded Output**
Large positive values may cause instability.

Softmax Activation Function

- The **Softmax activation function** is used in neural networks to convert a vector of values into **probability values**. The output probabilities range between **0 and 1**, and the **sum of all probabilities equals 1**.
- Softmax is mainly used in the **output layer of neural networks for multi-class classification problems**.
- Example: Classifying an image as **cat, dog, or bird**.

$$0 \leq f(x_i) \leq 1$$



$$f(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

x_i → input value for class i

e^{x_i} → exponential of input

n → number of classes

$f(x_i)$ → probability of class i

Working of Softmax Function

1. Take the input values (logits) from the neural network.
2. Apply the exponential function to each value.
3. Divide each exponential value by the sum of all exponential values.
4. The results become probabilities.

$$x = [2, 1, 0.1]$$

$$e^2 = 7.39, \quad e^1 = 2.72, \quad e^{0.1} = 1.105$$

$$7.39 + 2.72 + 1.105 = 11.215$$

$$P_1 = 7.39/11.215 = 0.659$$

$$P_2 = 2.72/11.215 = 0.242$$

$$P_3 = 1.105/11.215 = 0.099$$

Working of Softmax Function

Advantages

- Provides **probability distribution**
- Useful for **classification tasks**
- Easy interpretation of results
- Works well with **cross-entropy loss**

Disadvantages

- Computationally expensive due to exponential operations
- Sensitive to large input values
- Can cause **numerical instability** if inputs are very large

Feature	Sigmoid	Tanh	ReLU	Softmax
Mathematical Formula	$f(x) = \frac{1}{1+e^{-x}}$	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$f(x) = \max(0, x)$	$f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$
Output Range	0 to 1	-1 to 1	0 to ∞	0 to 1 (probabilities)
Shape of Curve	S-shaped curve	S-shaped curve	Piecewise linear	Exponential probability distribution
Computational Complexity	Moderate	Moderate	Very low	High
Speed of Training	Slow	Faster than sigmoid	Very fast	Moderate
Main Usage	Binary classification output layer	Hidden layers	Hidden layers in deep networks	Multi-class classification output layer
Advantages	Probabilistic output	Zero-centered outputs	Fast and efficient	Produces probability distribution
Disadvantages	Vanishing gradient, slow learning	Vanishing gradient	Dead neuron problem	Sensitive to large inputs

Underfitting

- **Underfitting** occurs when a model is **too simple to capture the underlying pattern of the data**. The model fails to learn the important features from the training dataset.
- As a result, the model performs **poorly on both training data and test data**.
 - High training error
 - High testing error
 - Model cannot learn data patterns
 - Poor predictive performance

Underfitting

Causes of Underfitting

- Model too simple
- Insufficient training time
- Too few features
- Improper hyperparameters
- Excessive regularization

How to Reduce Underfitting

- Increase model complexity
- Add more features
- Train for more epochs
- Reduce regularization
- Use a more powerful model

Overfitting

- **Overfitting** occurs when a model **learns the training data too well**, including noise and irrelevant details. The model performs very well on the **training data but poorly on new test data**.
- This means the model **memorizes the data instead of learning patterns**.
- **Characteristics of Overfitting**
 - Very low training error
 - High testing error
 - Model captures noise in data
 - Poor generalization

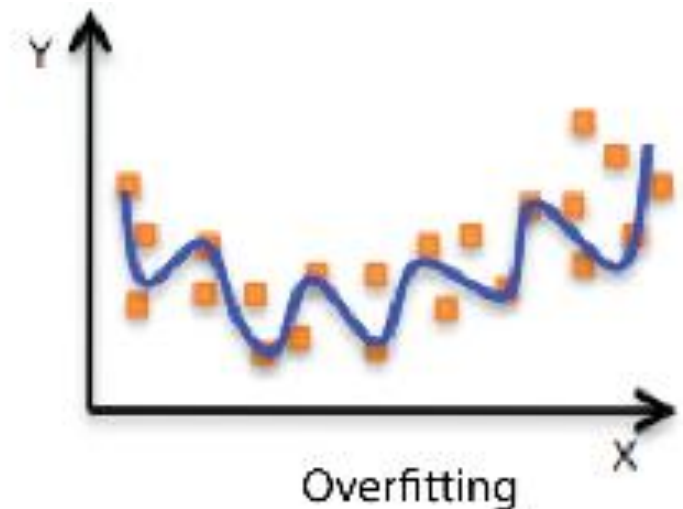
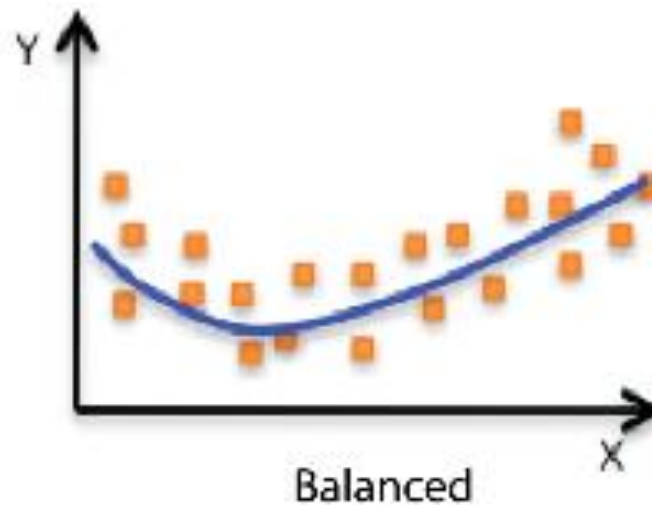
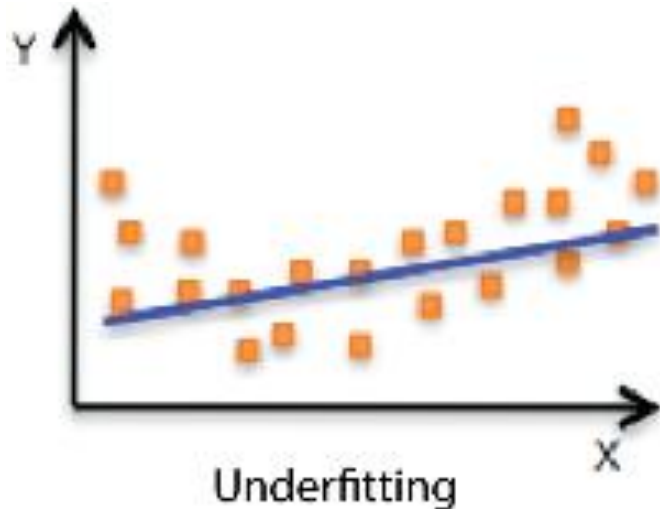
Overfitting

Causes of Overfitting

- Model too complex
- Too many parameters
- Small training dataset
- Too many training epochs
- Noise in training data

How to Reduce Overfitting

- Increase training data
- Use regularization techniques
- Apply dropout
- Use early stopping
- Reduce model complexity
- Perform cross-validation



Bias–Variance Tradeoff

- The **Bias–Variance Tradeoff** explains the relationship between **model complexity**, **prediction accuracy**, and **generalization ability**.
- It helps in understanding the problems of **underfitting** and **overfitting** and finding the **optimal model complexity**.

Bias–Variance Tradeoff

- **Bias** refers to the error caused by **simplifying assumptions made by the model** to learn the target function.
- A model with **high bias** pays very little attention to the training data and oversimplifies the model.
- **Characteristics of High Bias**
 - Model is too simple
 - Cannot capture complex patterns
 - Leads to **underfitting**
- **Effects**
 - High training error
 - High testing error

Bias–Variance Tradeoff

- **Variance** refers to the error caused by the model being **too sensitive to the training data**.
- A model with **high variance** learns not only the pattern but also the **noise present in the training dataset**.
- **Characteristics of High Variance**
 - Model is too complex
 - Sensitive to small changes in data
 - Leads to **overfitting**
- **Effects**
 - Low training error
 - High testing error

Bias–Variance Tradeoff

- There is a **tradeoff between bias and variance**:
 - Increasing model complexity **reduces bias but increases variance**
 - Decreasing model complexity **reduces variance but increases bias**
 - The goal is to **balance both to minimize total prediction error**.
-
- The prediction error of a model can be expressed as:

$$\textit{Total Error} = \textit{Bias}^2 + \textit{Variance} + \textit{Irreducible Error}$$

Bias² → Error due to incorrect model assumptions

Variance → Error due to sensitivity to training data

Irreducible error → Noise in data that cannot be removed

Bias–Variance Tradeoff

- The **Bias–Variance Tradeoff** explains how model complexity affects prediction accuracy.
- **High bias leads to underfitting**
- **High variance leads to overfitting.**
- The objective in machine learning is to achieve a balance between bias and variance to minimize prediction error and improve generalization.

Loss Function

A **loss function** (also called **cost function**) is a mathematical function that measures **how far the predicted output of a model is from the actual (true) output**.

- **Low loss** → Model predictions are accurate
- **High loss** → Model predictions are poor

$$L(y, \hat{y})$$

y → actual value

$\hat{y}$ → predicted value

L → loss function

Loss Function

Types of Loss Functions

- Regression Loss Functions
 - Classification Loss Functions
-
- **Regression Loss Functions:** Used when output is continuous.
 - **Classification Loss Functions:** Used when output is categorical.

Loss Function - Regression Loss Functions

Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Characteristics

- Squares the error
- Penalizes large errors more

Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Characteristics

- Uses absolute difference
- Less sensitive to outliers

Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Gives error in same units as output.

Loss Function - Classification Loss Functions

- **Binary Cross-Entropy (Log Loss):**
Used for **binary classification**.

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Use

- Sigmoid activation
- Binary classification problems

- **Categorical Cross-Entropy:**
Used for **multi-class classification**.

$$L = - \sum y_i \log(\hat{y}_i)$$

Use

- Softmax activation
- Multi-class problems

Deep learning libraries

- **Deep learning libraries** are software tools and frameworks that provide **pre-built functions, models, and utilities** to design, train, and deploy neural networks efficiently.
- They simplify complex tasks like:
 - Building neural networks
 - Training models
 - Handling large datasets
 - Performing mathematical computations

Deep learning libraries

- **TensorFlow**

- Open-source deep learning framework developed by Google
- Widely used for research and production

- **Features:**

- Supports CPU and GPU
- Scalable and flexible
- Used for large-scale applications

- **Applications:**

- Image recognition
- NLP
- Speech processing

- **Keras**

- High-level API built on top of TensorFlow
- User-friendly and easy to learn

- **Features:**

- Simple syntax
- Fast prototyping
- Ideal for beginners

- **Use:**

- Building and training neural networks quickly

Deep learning libraries

- **PyTorch**

- Developed by Meta (Facebook)
- Popular in research community

- **Features:**

- Dynamic computation graph
- Easy debugging
- Python-friendly

- **Applications:**

- Computer vision
- NLP
- Research experiments

- **MXNet**

- Flexible and scalable framework
- Supported by Apache

- **Features:**

- Multi-language support
- Efficient for distributed training

- **Scikit-learn**

- General machine learning library
- Not deep learning-specific but widely used

- **Features:**

- Data preprocessing
- Model evaluation
- Basic ML algorithms